HLD Level 2.1

Design a Centralized Logging System - Complete Interview Question & Solution

Interview prompt: Design a centralized logging system that collects logs from thousands of application servers, stores them reliably, makes recent logs searchable within seconds, and supports retention without allowing the logging infrastructure to harm production applications.

1. Clarify Requirements

Functional requirements

Collect logs from thousands of application servers; centralize them; search/filter by time, service, severity, host and request/correlation ID; support near-real-time viewing; retain logs for a configured period; and tolerate temporary downstream failures.

Non-functional requirements

High write throughput, horizontal scalability, high availability, durability appropriate to log priority, bounded memory/disk usage, predictable ingestion delay, and reasonable search latency. Perfect global ordering is normally unnecessary.

2. Capacity Estimation

```
Assume:
10,000 servers
100 log events/sec/server
500 bytes/log

Events/sec = 10,000 x 100 = 1,000,000 logs/sec
Bandwidth = 1,000,000 x 500 B = 500 MB/sec
Daily raw = 500 MB/sec x 86,400 ~= 43.2 TB/day
7 days raw ~= 302 TB
```

Actual storage depends on compression, indexes, replicas, metadata and retention tiers. The estimate immediately tells us a single server/database design is insufficient.

3. High-Level Architecture

```
Application Servers
|
Local Log Agents
|
Load Balancer
|
Ingestion Servers
|
Message Broker
|
Processing Workers
|
+----------------------+
| |
Search / Hot Store Object / Cold Store
|
Search API
|
Logging UI / Engineers
```

Core design decisions: isolate applications with local agents; decouple ingestion and storage with a broker; bound every buffer; use indexed hot storage for recent search and cheaper cold storage for retention.

4. Why Use a Local Log Agent?

Applications should normally write to stdout/files or a lightweight local logging path rather than synchronously depending on the central logging service. A local agent tails/receives logs, batches them, compresses them, temporarily buffers them and sends them upstream.

```
Application -> local file/stdout -> Agent -> Central Logging
```

If central logging becomes slow, the production application should not become slow merely because it is trying to emit logs.

5. Local Buffering

The agent can use a bounded local disk buffer during temporary network/central-service outages. It must not buffer forever: filling the host disk could damage the application itself.

> **When the buffer is full, use an explicit degradation policy. For example, sample/drop DEBUG first, then INFO, while preserving ERROR longer if requirements permit.**

6. Batching and Compression

Sending one network request per log is inefficient at very high rates. Agents can send when a batch reaches a size threshold or a short time threshold.

```
Example policy:
send when batch = 500 logs
OR 100 ms elapsed
```

Batching improves network/syscall/serialization efficiency at the cost of a small amount of ingestion latency.

7. Load Balancer and Ingestion Tier

```
+-> Ingestion 1
Agents -> LB -+-> Ingestion 2
+-> Ingestion N
```

Keep ingestion servers as stateless as practical so they can scale horizontally and failed instances can be removed through health/readiness checks.

8. Why Not Write Directly to Search Storage?

Direct synchronous writes couple ingestion capacity to storage latency. A traffic spike or slow search cluster can then block ingestion.

```
BAD coupling:
Agents -> Ingestion -> Search DB

Better:
Agents -> Ingestion -> Broker -> Consumers -> Storage
```

9. Message Broker

The broker absorbs temporary bursts, decouples ingestion from processing/storage, partitions work, supports independent consumer scaling and can provide retention/replay.

> **A broker does not solve permanent overload. If producers sustain 1M logs/sec and consumers sustain 800k/sec, backlog grows by 200k/sec until capacity is increased or load is reduced.**

10. Broker Partitioning

Partition the stream so multiple consumers can process logs in parallel. The partition key depends on ordering requirements and load distribution.

```
Ingestion -> P0
-> P1
-> P2
-> ... PN
```

Using only service_id can preserve per-service partition ordering but can create a hot partition if one service generates a disproportionate share of logs. Global ordering is normally not worth the scalability cost.

11. Processing Workers

Consumer workers parse and validate records, normalize fields, add metadata, extract searchable fields and batch storage writes.

```
Raw log -> parse -> normalize -> enrich -> batch -> storage
```

A structured record might include timestamp, service, host, severity, message, request_id, source and sequence/event ID.

12. Hot Search Storage

Recent logs need an indexed/searchable store because engineers commonly ask for ERROR logs from a service within a short time range or for a particular request ID.

Useful indexes commonly include timestamp plus selected high-value fields such as service, severity and request/correlation ID. Do not index every arbitrary field because indexes increase storage and write amplification.

13. Hot, Warm and Cold Retention

```
0-7 days -> Hot indexed/searchable
7-30 days -> Warm/reduced indexing
30-90 days -> Cold object storage
```

Keeping all historical logs in expensive search storage is usually unnecessary. Tiering separates low-latency recent investigation from cost-effective long-term retention.

14. Read Path vs Write Path

```
WRITE:
Apps -> Agents -> LB -> Ingestion -> Broker -> Workers -> Storage

READ:
Engineer -> UI -> Search API -> Search Store -> Results
```

Explicitly separating read and write paths is a strong HLD habit.

15. Search and Time Partitioning

Logs naturally contain timestamps. Organizing/indexing by time lets a query for 10:00-10:05 search only relevant ranges instead of scanning hundreds of terabytes.

16. Reliability - Ingestion Failure

If an ingestion server crashes, readiness/health checks remove it from routing. Agents reconnect/retry another healthy instance using bounded retries, exponential backoff and jitter.

17. Broker or Downstream Failure

During a temporary central outage, agents use bounded local buffering and ingestion uses bounded in-process queues. If downstream capacity does not recover, the system must throttle, reject, sample or drop lower-priority logs rather than consume unbounded memory/disk.

18. At-Least-Once and Duplicates

```
Agent sends batch -> broker accepts -> ACK lost -> agent retries -> duplicate possible
```

For logging, occasional duplicates may be preferable to loss. If duplicate suppression is required, attach an event_id or agent_id + sequence number and deduplicate where economically justified. Do not casually claim end-to-end exactly-once.

19. Ordering

Perfect ordering across thousands of hosts is usually unnecessary and difficult because clocks differ and logs arrive through parallel paths. Preserve source-local or partition-local ordering where useful and rely on timestamps/correlation IDs for investigation.

20. Backpressure

```
incoming = 1.5M/sec
processing = 1.0M/sec
backlog growth = 0.5M/sec
```

Monitor broker lag, queue depth, ingestion rate and consumer throughput. Responses include scaling consumers, improving batching, throttling noisy producers, sampling low-value logs or reducing log level.

21. Noisy Neighbor Protection

A broken service emitting hundreds of thousands of DEBUG logs per second must not starve critical logs from other services. Use per-service quotas, rate limits, sampling and priority-aware policies.

```
ERROR -> preserve strongly
WARN -> preserve
INFO -> normal policy
DEBUG -> sample/drop aggressively during overload
```

22. Security Considerations

Authenticate agents, encrypt traffic, authorize search access, protect sensitive fields, and define retention/redaction rules. Logs can contain tokens, personal data or secrets if applications log carelessly, so collection pipelines should discourage or filter sensitive content.

23. Observability of the Logging Platform

Monitor logs/sec, bytes/sec, ingestion delay, agent send failures, retry count, dropped/sampled logs, broker lag, consumer throughput, queue depth, storage utilization, indexing latency and search P50/P95/P99.

```
generated at 10:00:00
searchable at 10:00:03
end-to-end ingestion delay = 3 sec
```

24. C++ Ingestion Server Deep Dive

```
Many agent sockets
|
epoll / event loop
|
batch parser
|
bounded in-process queue
|
worker pool
|
broker producer
```

For tens of thousands of mostly idle or bursty connections, an event-driven networking layer can be preferable to one blocked thread per connection.

25. Bounded In-Process Queue

The queue between network intake and broker workers must be bounded. If broker publication slows, an unbounded queue converts the downstream problem into process memory exhaustion.

```
network thread -> [bounded queue] -> workers -> broker
```

26. Synchronization

A basic C++ implementation may use mutex + condition_variable for a work queue. Consumers sleep while empty and wake on producer notification rather than busy waiting. At HLD level, focus first on capacity/full policy; exact lock implementation belongs in the component deep dive.

27. Ownership and Copies

Large batches should have explicit ownership. Moveable buffers or unique_ptr can transfer ownership from network parsing to queue to worker without unnecessary repeated copies. shared_ptr should be used only for genuine shared ownership.

```
receive buffer -> move -> queue -> move -> worker
```

28. Graceful Shutdown

```
SIGTERM
-> mark NOT READY
-> stop new intake
-> drain/finish queued work by policy
-> flush broker producer
-> join workers
-> close sockets/resources
-> exit
```

This is an excellent C++ systems follow-up because it connects readiness, bounded queues, worker lifecycle, condition-variable wakeup and RAII.

29. Common Interview Follow-Ups

Why a broker? Decoupling, burst absorption, replay and independent scaling.

What if the broker fills? Scale/fix consumers, apply backpressure, quotas/sampling; buffering cannot solve permanent mismatch.

What if search storage fails? Continue ingestion through broker if capacity permits and catch up after recovery.

How do you prevent one service from flooding the system? Per-service quotas/rate limits and priority-aware sampling.

Do you need global ordering? Usually no; local/partition ordering plus timestamps is sufficient.

How do you avoid losing logs? Local bounded disk buffering, durable broker semantics appropriate to priority, retries and replicated storage.

How do you avoid duplicates? Event IDs/sequence numbers when required; otherwise accept at-least-once duplicates if that trade-off is preferable to loss.

30. Bottlenecks to Discuss

Potential bottlenecks include agent network bandwidth, ingestion CPU/network, broker partitions, hot partition keys, consumer throughput, indexing/write amplification, search fan-out, storage capacity, large/high-cardinality fields and retry/reconnect storms.

31. Five Core Decisions to Remember

1. **Application should not synchronously depend on central logging -> LOCAL AGENT.**
2. **Decouple ingestion from storage and absorb temporary bursts -> BROKER.**
3. **Never allow slowdown to become unlimited memory/disk -> BOUNDED BUFFERS/QUEUES.**
4. **Recent search and cheap retention have different needs -> HOT + COLD STORAGE.**
5. **One noisy producer must not harm everyone -> QUOTAS / RATE LIMIT / SAMPLING.**

32. Architecture to Draw From Memory

```
Apps
|
Agents
|
Load Balancer
|
Ingestion Tier
|
Message Broker
|
Consumer Workers
|
+--------------------+
| |
Hot Search Store Cold Object Store
|
Search API
|
UI
```

33. Interview-Ready Spoken Answer

"First I'd clarify scale, retention, acceptable ingestion delay, search patterns and durability. I'll assume roughly one million logs per second and that recent logs should become searchable within a few seconds. Applications should not

synchronously depend on the logging backend, so I'd run lightweight local agents that batch, compress and temporarily buffer logs. Agents send through a load balancer to horizontally scalable stateless ingestion servers. Rather than writing synchronously to search storage, ingestion publishes to a partitioned durable broker, which absorbs temporary bursts and decouples downstream processing. Consumer workers parse and batch logs into indexed hot storage for recent search, while older logs move to cheaper object storage. I'd explicitly handle overload with bounded buffers, backpressure, quotas and priority-aware sampling, and monitor broker lag, queue depth, dropped logs, ingestion delay and search latency. For the C++ ingestion tier, I'd consider event-driven socket handling, bounded worker queues, explicit buffer ownership and graceful shutdown."

34. Final Interview Checklist

Requirements -> capacity -> agents -> batching -> LB -> stateless ingestion -> broker -> partitioning -> workers -> hot/cold storage -> read path -> reliability -> duplicates/order -> backpressure -> noisy neighbors -> observability -> C++ worker/network/ownership/shutdown deep dive.